

Балтийский государственный технический университет
«ВОЕНМЕХ» им. Д. Ф. Устинова

Кафедра О7 «Информационные системы и программная инженерия»

Практическая работа №3
по дисциплине «Структуры и организация данных»
на тему «Оценка эффективности алгоритмов»
часть 2 «Алгоритмы поиска»

вариант 12

Выполнил:

Студент:

Праудин Д. С .

Группа О717Б

Преподаватель:

Палехова О. А.

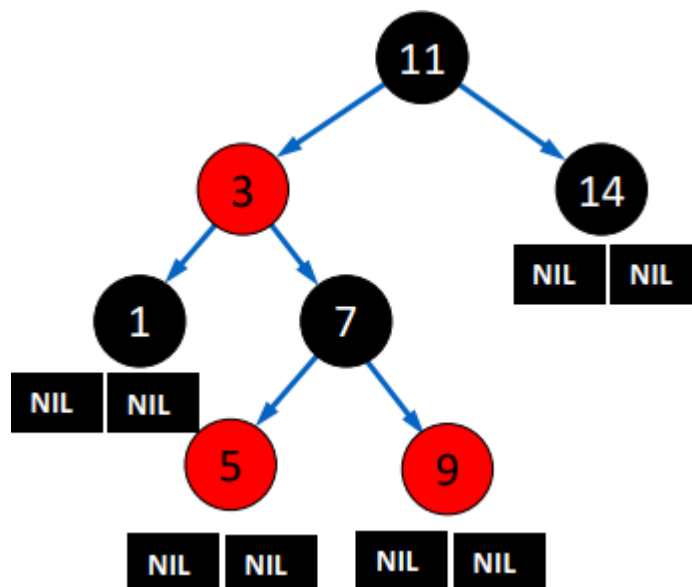
Санкт-Петербург
2022 г.

Произвести сравнение указанных в вариативной части структур данных по пространственной сложности и вычислительной сложности поиска.

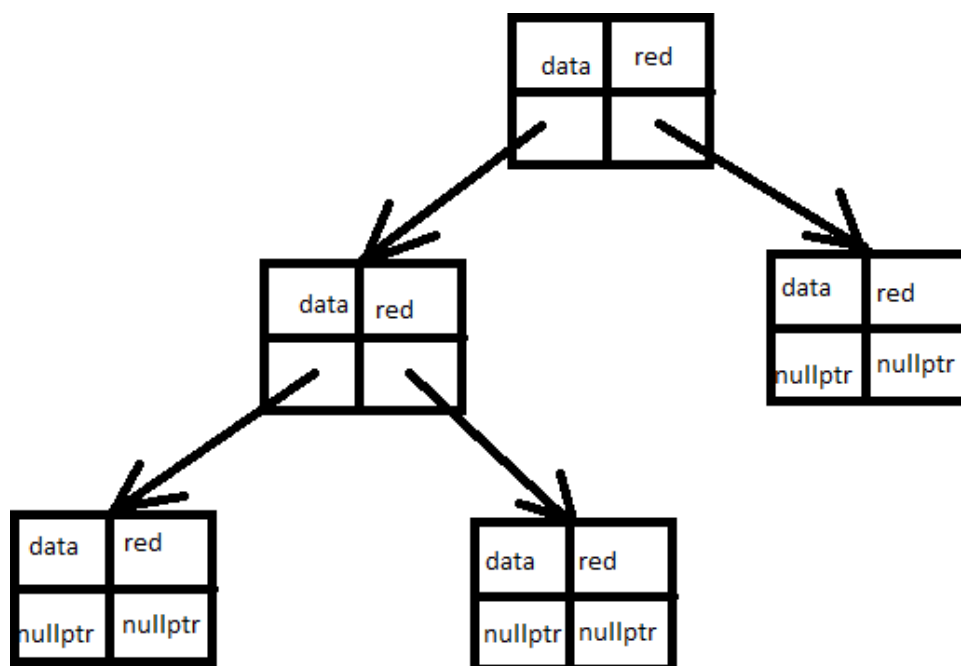
Выполнить теоретические расчеты сложности поиска и занимаемого структурой объема памяти. Сформировать корректный тестовый набор из 100 ключей. Реализовать две указанные структуры данных, заполнив их значениями из приложенного файла *test_numbers.txt*. Выполнить поиск 100 ключей в указанных структурах данных, для каждого ключа выводить сообщение о том, найден он или нет, и количество выполненных при поиске сравнений ключей, в конце программы вывести среднее количество сравнений, пришедшееся на один ключ. Произвести анализ полученных теоретических и практических результатов, сравнить поисковые структуры по вычислительной и пространственной сложности и выбрать наиболее эффективную из них.

Красно-черное дерево, рандомизированное бинарное дерево поиска.

Схематичное изображение красно-черного дерева:



Схематичное изображение структуры хранения, использованной в программе для программирования красно-черного дерева:



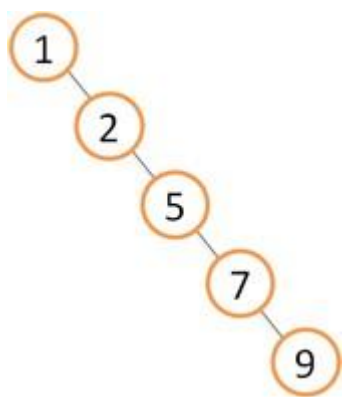
(data – ключ, red – переменная, хранящая признак “красноты” узла)

Трудоемкость поиска:

Случай	Ситуация, соответствующая случаю	Обоснование	Ожидаемое число сравнений при поиске	Асимптотическая оценка сложности поиска
наилучший	искомый ключ находится в корне дерева	требуется выполнить одно сравнение	1	$\Omega(1)$
наихудший	искомый ключ отсутствует в дереве, при этом его поиск идет по самой длинной ветке	требуется осуществить перебор ключей на одной ветке дерева два сравнения на $\log_2 N$	$2 * \log_2 N$	$O(\log N)$
наиболее вероятный	искомый ключ присутствует в дереве и находится в его середине	требуется осуществить перебор ключей на одной ветке дерева средняя высота дерева	$\log_2 N$	$O(\log N)$

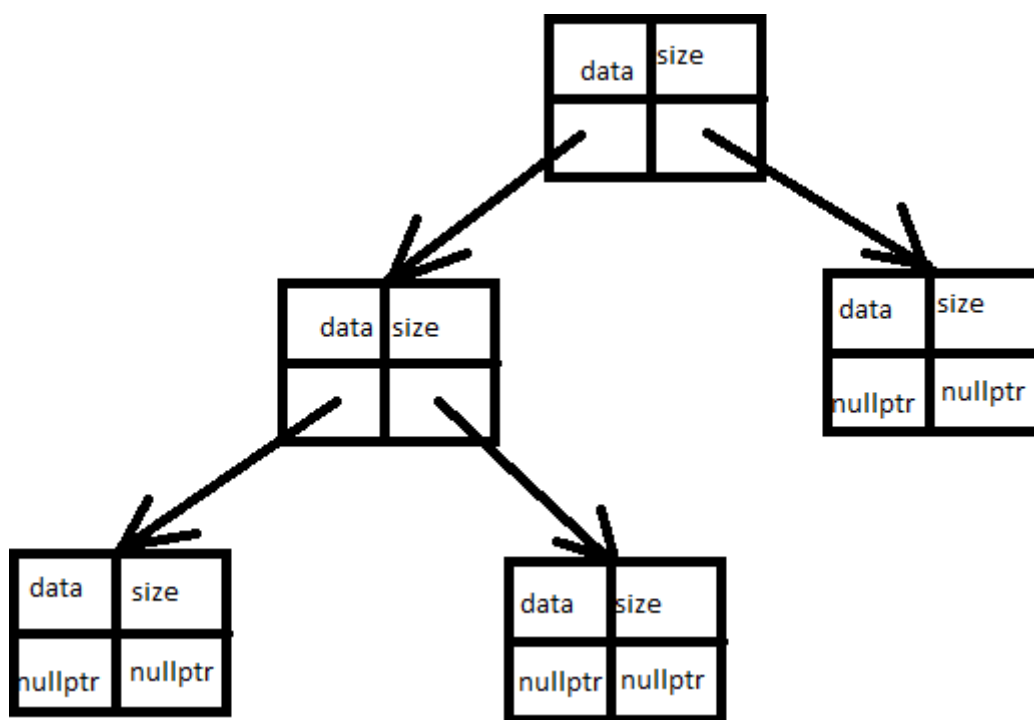
Требуемый объем памяти $(2 * \text{sizeof}(\text{int}) + 2 * \text{sizeof}(\text{pointer})) * N = (8 + 2 * 8) * 200001 = 4800024$ байт.

Схематичное изображение рандомизированного бинарного дерева поиска:



(одна из возможных схем данного дерева, но в реальности такая схема почти невозможна)

Схематичное изображение структуры хранения, использованной в программе для программирования рандомизированного бинарного дерева поиска:



(data – ключ, size – переменная, хранящая количество узлов текущего поддерева)

Трудоёмкость поиска:

Случай	Ситуация, соответствующая случаю	Обоснование	Ожидаемое число сравнений при поиске	Асимптотическая оценка сложности поиска
наилучший	искомый ключ находится в корне дерева	требуется выполнить одно сравнение	1	$\Omega(1)$
наихудший	искомый ключ отсутствует в дереве, при этом его поиск идет по	требуется осуществить перебор ключей на одной ветке дерева два сравнения на $2 * \log_2 N$	$4 * \log_2 N$	$O(\log N)$

	самой длинной ветке			
наиболее вероятный	искомый ключ присутствует в дереве и находится в его середине	требуется осуществить перебор ключей на одной ветке дерева средняя высота дерева	$2 * \log_2 N$	$O(\log N)$

Требуемый объем памяти $(2 * \text{sizeof}(\text{int}) + 2 * \text{sizeof}(\text{pointer})) * N = (8 + 2 * 8) * 200001 = 4800024$ байт.

Тестовый набор ключей:

присутствующие в файле: 97192338, 90229668, 67343291, 70258238, 83246999, 37801902, 30349033, 19445301, 88278989, 36406281, 35966130, 35557396, 84580155, 83130037, 42315857, 15916697, 52779136, 32051522, 39771580, 20138140, 42947553, 95905677, 62667096, 66541010, 30617946, 36600568, 99020052, 55743512, 83673374, 42882723, 56804407, 66912047, 97986428, 56718265.

отсутствующие в файле: 18478-меньше, 22772-меньше, 17106-меньше, 20122-меньше, 13638-меньше, 28836-меньше, 2069-меньше, 24085-меньше, 23625-меньше, 8573-меньше, 4386-меньше, 18532-меньше, 1087-меньше, 3896-меньше, 18528-меньше, 3488-меньше, 12011-меньше, 28247-меньше, 8917-меньше, 22278-меньше, 1955-меньше, 1966-меньше, 9626-меньше, 13984-меньше, 13407-меньше, 5409-меньше, 24460-меньше, 2594-меньше, 15631-меньше, 23471-меньше, 11077-меньше, 1298-меньше, 32050-меньше, 300000001-больше, 10065409-больше, 2705032705-больше, 1100000001-больше, 3700000001-больше, 2900000001-больше, 2805032705-больше, 600000001-больше, 500000001-больше, 3900000001-больше, 210065409-больше, 205032705-больше, 500000001-больше, 2700000001-больше, 3200000001-больше, 3005032705-больше, 610065409-больше, 3905032705-больше, 1800000001-больше, 900000001-больше, 3000000001-больше, 1105032705-больше, 4100000001-больше, 505032705-больше, 4000000001-больше, 2000000001-больше, 4205032705-больше, 3200000001-больше, 510065409-больше, 1700000001-больше, 2300000001-больше, 310065409-больше, 3805032705-больше.

Текст программы:

```
#include <iostream>
#include <windows.h>
#include <fstream>
#include <ctime>

#define A 10000000
#define B 100000000
#define C 100
#define FS "D:\\Programms\\SIOD\\Pr3\\pr2\\cmake-build-debug\\set.txt"

using namespace std;
```

```

/* структура, описывающая узел красно-черного дерева */
struct rb_node
{
    int red;
    unsigned int data;
    struct rb_node *link[2];
};

struct rb_tree
{
    struct rb_node *root; // указатель на корневой узел
    int count; // количество узлов в дереве
};

/* узел рандомизированного дерева */
struct node {
    unsigned int key;
    int size;
    node *left, *right;
    node(int k) {
        key = k;
        left = right = nullptr;
        size = 1;
    }
};

void CreateSet(const char *); // функция для создания набора
struct rb_tree * tree_create(); // создание черно-красного дерева
int is_red ( struct rb_node *); // проверка на красноту
struct rb_node *rb_single ( struct rb_node *, int); // одинарный поворот
struct rb_node *rb_double ( struct rb_node *, int ); // двойной поворот
struct rb_node *make_node ( int ); // сделать узел дерева
int rb_insert ( struct rb_tree *, unsigned int ); // вставка элемента в узел
дерева
void del_rb_tree (struct rb_node *); // удаление дерева
void FillRb(const char*, struct rb_tree*); // заполнение дерева
void CheckRb(struct rb_tree *, unsigned int **, bool ); // прогонка красно
черного дерева
bool SearchEl(unsigned int, struct rb_node *, unsigned int *); // поиск
элемента
int getsize (node* ); // обертка поля size
void fixsize (node* ); // установление корректного размера дерева
node* rotateright (node* ); // правый поворот вокруг узла p
node* rotateleft (node* ); // левый поворот вокруг узла p
node* insertroot (node* , int ); // вставка нового узла с ключом k в корень
дерева p
node* insert (node* , int ); // рандомизированная вставка нового узла с ключом
k в дерево p
void del_rn_tree (struct node *); // удаление рандомизированного дерева
struct node* FillRn(const char *, struct node *); // заполнение
рандомизированного дерева
void CheckRn(struct node *, unsigned int **, bool ); // проверка
рандомизированного дерева
bool SearchElRand(unsigned int , struct node *, unsigned int *); // поиск узла
в рандомизированном дереве

int main(int argc, char *argv[]) {
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    srand(time(NULL));
    char filename[80];

```

```

bool fl[2];
fl[0] = false;
fl[1] = false;
unsigned int **cmp;
double sr[2] = {0};
cmp = new unsigned int * [2];
cmp[0] = new unsigned int[C];
cmp[1] = new unsigned int[C];
int menu, t;
do{
    do {
        system("cls");
        cout << "1.Пересоздать набор\n2.Выполнить проверку красно-чёрного
дерева\n3.Выполнить проверку рандомизированного дерева\n4.Выход\n";
        t = scanf("%d", &menu);
        while(cin.get() != '\n');
    }while(t != 1 || menu < 1 || menu > 4);
    switch(menu){
        case 1:{
            system("cls");
            if(argc > 1){
                strcpy(filename, argv[1]);
            }
            else{
                cout << "Введите имя файла: ";
                cin >> filename;
            }
            try{
                CreateSet(filename);
            }catch (const char *msg){
                cout << msg << endl;
                system("pause");
            }
        }break;
        case 2:{
            system("cls");
            struct rb_tree *rb = tree_create();
            if(argc > 1){
                strcpy(filename, argv[1]);
            }
            else{
                cout << "Введите имя файла: ";
                cin >> filename;
            }
            try{
                FillRb(filename, rb);
            }catch (const char *msg){
                cout << msg << endl;
                system("pause");
            }
            CheckRb(rb, cmp, fl[0]);
            fl[0] = true;
            system("pause");
            del_rb_tree(rb->root);
        }break;
        case 3:{
            system("cls");
            struct node *rn = nullptr;
            if(argc > 1){
                strcpy(filename, argv[1]);
            }
            else{
                cout << "Введите имя файла: ";

```

```

        cin >> filename;
    }
    try{
        rn = FillRn(filename, rn);
    }catch (const char *msg){
        cout << msg << endl;
        system("pause");
    }
    CheckRn(rn, cmp, fl[1]);
    fl[1] = true;
    system("pause");
    del_rn_tree(rn);
}break;
}
}while(menu != 4);
system("cls");
if(fl[0] && fl[1]) {
    for (int i = 0; i < C; i++) {
        cout << "Ключ " << i + 1 << " [RB/Rnd]: " << cmp[0][i] << '/' <<
cmp[1][i] << endl;
        sr[0] += (double)cmp[0][i];
        sr[1] += (double)cmp[1][i];
    }
    sr[0] /= C;
    sr[1] /= C;
    cout << "Среднее [RB/Rnd]: " << sr[0] << "/" << sr[1] << endl;
    system("pause");
}
delete[] cmp[0];
delete[] cmp[1];
delete[] cmp;
return 0;
}

void CreateSet(const char *fname){
    ifstream f(fname);
    if(!f.is_open())
        throw "Ой, а файла тут нет =/\n";
    ofstream fl(FS);
    unsigned int x;
    int i1 = 0, i2 = 0, i3 = 0;
    while(i1 + i2 + i3 != C){
        f >> x;
        if(x >= A && x < B && i2 <= C / 3) {
            fl << x << ' ';
            i2++;
        }
        else if(x < A && i1 < C / 3){
            fl << x << ' ';
            i1++;
        }
        else if(x > B && i3 < C / 3){
            fl << x << ' ';
            i3++;
        }
        else if(i1 < C / 3){
            x = rand() % A + 1;
            fl << x << ' ';
            i1++;
        }
        else{
            x = rand() % 100 * B + 1;
            fl << x << ' ';
        }
    }
}

```



```

        i3++;
    }
    cout << i1 << ' ' << i2 << ' ' << i3 << endl;
}
f.close();
fl.close();
cout << "Набор успешно создан!\n";
system("pause");
}

struct rb_tree * tree_create()
{
    struct rb_tree * my_tree = new struct rb_tree;
    my_tree->root = nullptr;
    my_tree->count = 0;
    return my_tree;
}

int is_red ( struct rb_node *node )
{
    return node != nullptr && node->red == 1;
}

/* функция для однократного поворота узла */
struct rb_node *rb_single ( struct rb_node *root, int dir )
{
    struct rb_node *save = root->link[!dir];

    root->link[!dir] = save->link[dir];
    save->link[dir] = root;

    root->red = 1;
    save->red = 0;

    return save;
}

/* функция для двукратного поворота узла */
struct rb_node *rb_double ( struct rb_node *root, int dir )
{
    root->link[!dir] = rb_single ( root->link[!dir], !dir );
    return rb_single ( root, dir );
}

struct rb_node *make_node ( int data )
{
    struct rb_node *rn = new struct rb_node;

    if ( rn != nullptr ) {
        rn->data = data;
        rn->red = 1; /* -инициализация красным цветом */
        rn->link[0] = nullptr;
        rn->link[1] = nullptr;
    }
    return rn;
}

int rb_insert ( struct rb_tree *tree, unsigned int data )
{
    /* если добавляемый элемент оказывается первым - то ничего делать не
    нужно*/
    if ( tree->root == nullptr ) {
        tree->root = make_node ( data );
    }
}

```

```

        if ( tree->root == nullptr )
            return 0;
    }
    else {
        struct rb_node head = {0}; /* временный корень дерева */
        struct rb_node *g, *t;      /* дедушка и родитель */
        struct rb_node *p, *q;      /* родитель и итератор */
        int dir = 0, last;

        /* вспомогательные переменные */
        t = &head;
        g = p = nullptr;
        q = t->link[1] = tree->root;

        /* основной цикл прохода по дереву */
        for ( ; ; )
        {
            if ( q == nullptr ) {
                /* вставка ноды */
                p->link[dir] = q = make_node ( data );
                tree->count ++ ;
                if ( q == nullptr )
                    return 0;
            }
            else if ( is_red ( q->link[0] ) && is_red ( q->link[1] ) )
            {
                /* смена цвета */
                q->red = 1;
                q->link[0]->red = 0;
                q->link[1]->red = 0;
            }
            /* совпадение 2-х красных цветов */
            if ( is_red ( q ) && is_red ( p ) )
            {
                int dir2 = t->link[1] == g;

                if ( q == p->link[last] )
                    t->link[dir2] = rb_single ( g, !last );
                else
                    t->link[dir2] = rb_double ( g, !last );
            }

            /* такой узел в дереве уже есть - выход из функции */
            if ( q->data == data )
                break;

            last = dir;
            dir = q->data < data;

            if ( g != nullptr )
                t = g;
            g = p, p = q;
            q = q->link[dir];
        }

        /* обновить указатель на корень дерева */
        tree->root = head.link[1];
    }
    /* сделать корень дерева черным */
    tree->root->red = 0;

    return 1;
}

```

```

void del_rb_tree (struct rb_node *node)
{
    if (node->link[1]) del_rb_tree (node->link[1]);
    if (node->link[0]) del_rb_tree (node->link[0]);
    delete node;
}

void FillRb(const char *fname, struct rb_tree *t){
    ifstream f(fname);
    if(!f.is_open())
        throw "Ошибка при открытии файла :|\n";
    unsigned int x;
    while(!f.eof()){
        f >> x;
        rb_insert(t, x);
    }
    cout << "Дерево заполнено" << endl;
}

void CheckRb(struct rb_tree *t, unsigned int **arr, bool r){
    ifstream f(FS);
    unsigned int x, col;
    bool is;
    for(int i = 0; i < C; i++) {
        struct rb_node *cur = t->root;
        col = 0;
        f >> x;
        is = SearchEl(x, cur, &col);
        if(is){
            cout << i + 1 << " ключ в дереве найден, " << col << " сравнений"
<< endl;
        }
        else{
            cout << i + 1 << " ключ в дереве не найден, " << col << "
сравнений" << endl;
        }
        if(!r)
            arr[0][i] = col;
    }
}

bool SearchEl(unsigned int el, struct rb_node *c, unsigned int *k){
    for(;;){
        if(c == nullptr)
            return false;
        if(el == c->data){
            *k += 1;
            return true;
        }
        else if(el > c->data){
            *k += 1;
            c = c->link[1];
        }
        else if(el < c->data){
            *k += 1;
            c = c->link[0];
        }
    }
}

int getsize (node* p) // обертка для поля size
{

```

```

        if( !p ) return 0;
        return p->size;
    }

void fixsize (node* p) // установление корректного размера дерева
{
    p->size = getsize(p->left)+getsize(p->right)+1;
}

node* rotateright (node* p) // правый поворот вокруг узла p
{
    node* q = p->left;
    if( !q ) return p;
    p->left = q->right;
    q->right = p;
    q->size = p->size;
    fixsize(p);
    return q;
}

node* rotateleft (node* p) // левый поворот вокруг узла p
{
    node* q = p->right;
    if( !q ) return p;
    p->right = q->left;
    q->left = p;
    q->size = p->size;
    fixsize(p);
    return q;
}

node* insertroot (node* p, unsigned int k) // вставка нового узла с ключом k
в корень дерева p
{
    if( !p ) return new node(k);
    if( k<p->key )
    {
        p->left = insertroot(p->left,k);
        return rotateright(p);
    }
    else
    {
        p->right = insertroot(p->right,k);
        return rotateleft(p);
    }
}

node* insert (node* p, unsigned int k) // рандомизированная вставка нового
узла с ключом k в дерево p
{
    if( !p ) return new node(k);
    if( rand()%(p->size+1)==0 )
        return insertroot(p,k);
    if( p->key>k )
        p->left = insert(p->left,k);
    else
        p->right = insert(p->right,k);
    fixsize(p);
    return p;
}

void del_rn_tree (struct node *n)
{

```

```

        if (n->left) del_rn_tree (n->left);
        if (n->right) del_rn_tree (n->right);
        delete n;
    }

    struct node* FillRn(const char *fname, struct node *n){
        ifstream f(fname);
        if(!f.is_open())
            throw "Ошибка при открытии файла :|\n";
        unsigned int x;
        while(!f.eof()){
            f >> x;
            n = insert(n, x);
        }
        cout << "Дерево заполнено" << endl;
        return n;
    }

    void CheckRn(struct node *t, unsigned int **arr, bool r){
        ifstream f(FS);
        unsigned int x, col;
        bool is;
        for(int i = 0; i < C; i++) {
            struct node *cur = t;
            col = 0;
            f >> x;
            is = SearchElRand(x, cur, &col);
            if(is){
                cout << i + 1 << " ключ в дереве найден, " << col << " сравнений"
<< endl;
            }
            else{
                cout << i + 1 << " ключ в дереве не найден, " << col << "
сравнений" << endl;
            }
            if(!r)
                arr[1][i] = col;
        }
    }

    bool SearchElRand(unsigned int el, struct node *c, unsigned int *k){
        for(;;){
            if(c == nullptr)
                return false;
            if(el == c->key){
                *k += 1;
                return true;
            }
            else if(el > c->key){
                *k += 1;
                c = c->right;
            }
            else if(el < c->key){
                *k += 1;
                c = c->left;
            }
        }
    }
}

```

Результаты работы программы:

Ключ	1	[RB/Rnd]	: 6/27
Ключ	2	[RB/Rnd]	: 4/24
Ключ	3	[RB/Rnd]	: 7/25
Ключ	4	[RB/Rnd]	: 2/24
Ключ	5	[RB/Rnd]	: 8/19
Ключ	6	[RB/Rnd]	: 5/20
Ключ	7	[RB/Rnd]	: 4/25
Ключ	8	[RB/Rnd]	: 7/13
Ключ	9	[RB/Rnd]	: 9/13
Ключ	10	[RB/Rnd]	: 12/26
Ключ	11	[RB/Rnd]	: 2/18
Ключ	12	[RB/Rnd]	: 10/22
Ключ	13	[RB/Rnd]	: 6/14
Ключ	14	[RB/Rnd]	: 12/18
Ключ	15	[RB/Rnd]	: 6/26
Ключ	16	[RB/Rnd]	: 7/20
Ключ	17	[RB/Rnd]	: 1/17
Ключ	18	[RB/Rnd]	: 6/32
Ключ	19	[RB/Rnd]	: 7/26
Ключ	20	[RB/Rnd]	: 10/20
Ключ	21	[RB/Rnd]	: 12/26
Ключ	22	[RB/Rnd]	: 9/24
Ключ	23	[RB/Rnd]	: 7/30
Ключ	24	[RB/Rnd]	: 5/28
Ключ	25	[RB/Rnd]	: 9/20
Ключ	26	[RB/Rnd]	: 8/16
Ключ	27	[RB/Rnd]	: 14/23
Ключ	28	[RB/Rnd]	: 12/22
Ключ	29	[RB/Rnd]	: 15/21
Ключ	30	[RB/Rnd]	: 8/26
Ключ	31	[RB/Rnd]	: 4/27
Ключ	32	[RB/Rnd]	: 12/16

Ключ	31	[RB/Rnd]:	4/27
Ключ	32	[RB/Rnd]:	12/16
Ключ	33	[RB/Rnd]:	12/11
Ключ	34	[RB/Rnd]:	10/22
Ключ	35	[RB/Rnd]:	16/16
Ключ	36	[RB/Rnd]:	16/16
Ключ	37	[RB/Rnd]:	16/16
Ключ	38	[RB/Rnd]:	16/16
Ключ	39	[RB/Rnd]:	16/16
Ключ	40	[RB/Rnd]:	16/16
Ключ	41	[RB/Rnd]:	16/16
Ключ	42	[RB/Rnd]:	16/16
Ключ	43	[RB/Rnd]:	16/16
Ключ	44	[RB/Rnd]:	16/16
Ключ	45	[RB/Rnd]:	16/16
Ключ	46	[RB/Rnd]:	16/16
Ключ	47	[RB/Rnd]:	16/16
Ключ	48	[RB/Rnd]:	16/16
Ключ	49	[RB/Rnd]:	16/16
Ключ	50	[RB/Rnd]:	16/16
Ключ	51	[RB/Rnd]:	16/16
Ключ	52	[RB/Rnd]:	16/16
Ключ	53	[RB/Rnd]:	16/16
Ключ	54	[RB/Rnd]:	16/16
Ключ	55	[RB/Rnd]:	16/16
Ключ	56	[RB/Rnd]:	16/16
Ключ	57	[RB/Rnd]:	16/16
Ключ	58	[RB/Rnd]:	16/16
Ключ	59	[RB/Rnd]:	16/16
Ключ	60	[RB/Rnd]:	16/16
Ключ	61	[RB/Rnd]:	16/16
Ключ	62	[RB/Rnd]:	16/16

```
Ключ 61 [RB/Rnd]: 16/16
Ключ 62 [RB/Rnd]: 16/16
Ключ 63 [RB/Rnd]: 16/16
Ключ 64 [RB/Rnd]: 16/16
Ключ 65 [RB/Rnd]: 16/16
Ключ 66 [RB/Rnd]: 16/16
Ключ 67 [RB/Rnd]: 16/16
Ключ 68 [RB/Rnd]: 19/11
Ключ 69 [RB/Rnd]: 17/19
Ключ 70 [RB/Rnd]: 19/11
Ключ 71 [RB/Rnd]: 19/11
Ключ 72 [RB/Rnd]: 19/11
Ключ 73 [RB/Rnd]: 19/11
Ключ 74 [RB/Rnd]: 19/11
Ключ 75 [RB/Rnd]: 19/11
Ключ 76 [RB/Rnd]: 19/11
Ключ 77 [RB/Rnd]: 19/11
Ключ 78 [RB/Rnd]: 19/11
Ключ 79 [RB/Rnd]: 19/11
Ключ 80 [RB/Rnd]: 19/11
Ключ 81 [RB/Rnd]: 19/11
Ключ 82 [RB/Rnd]: 19/11
Ключ 83 [RB/Rnd]: 19/11
Ключ 84 [RB/Rnd]: 19/11
Ключ 85 [RB/Rnd]: 19/11
Ключ 86 [RB/Rnd]: 19/11
Ключ 87 [RB/Rnd]: 19/11
Ключ 88 [RB/Rnd]: 19/11
Ключ 89 [RB/Rnd]: 19/11
Ключ 90 [RB/Rnd]: 19/11
Ключ 91 [RB/Rnd]: 19/11
Ключ 92 [RB/Rnd]: 19/11
```

```
Ключ 93 [RB/Rnd]: 19/11
Ключ 94 [RB/Rnd]: 19/11
Ключ 95 [RB/Rnd]: 19/11
Ключ 96 [RB/Rnd]: 19/11
Ключ 97 [RB/Rnd]: 19/11
Ключ 98 [RB/Rnd]: 19/11
Ключ 99 [RB/Rnd]: 19/11
Ключ 100 [RB/Rnd]: 19/11
Среднее [RB/Rnd]: 14.21/16.4
```

Скриншоты срабатывания программы, на которых указано через '/' количество сравнений при поиске по дереву ключа и их среднее значение.

Дерево заполнено

1	ключ в дереве найден,	20	сравнений
2	ключ в дереве найден,	27	сравнений
3	ключ в дереве найден,	26	сравнений
4	ключ в дереве найден,	22	сравнений
5	ключ в дереве найден,	23	сравнений
6	ключ в дереве найден,	31	сравнений
7	ключ в дереве найден,	27	сравнений
8	ключ в дереве найден,	23	сравнений
9	ключ в дереве найден,	17	сравнений
10	ключ в дереве найден,	22	сравнений
11	ключ в дереве найден,	22	сравнений
12	ключ в дереве найден,	21	сравнений
13	ключ в дереве найден,	21	сравнений
14	ключ в дереве найден,	25	сравнений
15	ключ в дереве найден,	29	сравнений
16	ключ в дереве найден,	25	сравнений
17	ключ в дереве найден,	16	сравнений
18	ключ в дереве найден,	24	сравнений
19	ключ в дереве найден,	25	сравнений
20	ключ в дереве найден,	28	сравнений
21	ключ в дереве найден,	25	сравнений
22	ключ в дереве найден,	23	сравнений
23	ключ в дереве найден,	26	сравнений
24	ключ в дереве найден,	27	сравнений
25	ключ в дереве найден,	22	сравнений
26	ключ в дереве найден,	23	сравнений
27	ключ в дереве найден,	16	сравнений
28	ключ в дереве найден,	19	сравнений
29	ключ в дереве найден,	21	сравнений
30	ключ в дереве найден,	26	сравнений
31	ключ в дереве найден,	30	сравнений

[illegible]

```
69 ключ в дереве не найден, 20 сравнений
70 ключ в дереве не найден, 14 сравнений
71 ключ в дереве не найден, 14 сравнений
72 ключ в дереве не найден, 14 сравнений
73 ключ в дереве не найден, 14 сравнений
74 ключ в дереве не найден, 14 сравнений
75 ключ в дереве не найден, 14 сравнений
76 ключ в дереве не найден, 14 сравнений
77 ключ в дереве не найден, 14 сравнений
78 ключ в дереве не найден, 14 сравнений
79 ключ в дереве не найден, 14 сравнений
80 ключ в дереве не найден, 14 сравнений
81 ключ в дереве не найден, 14 сравнений
82 ключ в дереве не найден, 14 сравнений
83 ключ в дереве не найден, 14 сравнений
84 ключ в дереве не найден, 14 сравнений
85 ключ в дереве не найден, 14 сравнений
86 ключ в дереве не найден, 14 сравнений
87 ключ в дереве не найден, 14 сравнений
88 ключ в дереве не найден, 14 сравнений
89 ключ в дереве не найден, 14 сравнений
90 ключ в дереве не найден, 14 сравнений
91 ключ в дереве не найден, 14 сравнений
92 ключ в дереве не найден, 14 сравнений
93 ключ в дереве не найден, 14 сравнений
94 ключ в дереве не найден, 14 сравнений
95 ключ в дереве не найден, 14 сравнений
96 ключ в дереве не найден, 14 сравнений
97 ключ в дереве не найден, 14 сравнений
98 ключ в дереве не найден, 14 сравнений
99 ключ в дереве не найден, 14 сравнений
100 ключ в дереве не найден, 14 сравнений
```

Скриншот срабатывания программы при проверке только рандомизированного
дерева

[illegible]

```
51 ключ в дереве не найден, 16 сравнений
52 ключ в дереве не найден, 16 сравнений
53 ключ в дереве не найден, 16 сравнений
54 ключ в дереве не найден, 16 сравнений
55 ключ в дереве не найден, 16 сравнений
56 ключ в дереве не найден, 16 сравнений
57 ключ в дереве не найден, 16 сравнений
58 ключ в дереве не найден, 16 сравнений
59 ключ в дереве не найден, 16 сравнений
60 ключ в дереве не найден, 16 сравнений
61 ключ в дереве не найден, 16 сравнений
62 ключ в дереве не найден, 16 сравнений
63 ключ в дереве не найден, 16 сравнений
64 ключ в дереве не найден, 16 сравнений
65 ключ в дереве не найден, 16 сравнений
66 ключ в дереве не найден, 16 сравнений
67 ключ в дереве не найден, 16 сравнений
68 ключ в дереве не найден, 19 сравнений
69 ключ в дереве не найден, 17 сравнений
70 ключ в дереве не найден, 19 сравнений
71 ключ в дереве не найден, 19 сравнений
72 ключ в дереве не найден, 19 сравнений
73 ключ в дереве не найден, 19 сравнений
74 ключ в дереве не найден, 19 сравнений
75 ключ в дереве не найден, 19 сравнений
76 ключ в дереве не найден, 19 сравнений
77 ключ в дереве не найден, 19 сравнений
78 ключ в дереве не найден, 19 сравнений
79 ключ в дереве не найден, 19 сравнений
80 ключ в дереве не найден, 19 сравнений
81 ключ в дереве не найден, 19 сравнений
82 ключ в дереве не найден, 19 сравнений
83 ключ в дереве не найден, 19 сравнений
84 ключ в дереве не найден, 19 сравнений
85 ключ в дереве не найден, 19 сравнений
86 ключ в дереве не найден, 19 сравнений
87 ключ в дереве не найден, 19 сравнений
88 ключ в дереве не найден, 19 сравнений
89 ключ в дереве не найден, 19 сравнений
90 ключ в дереве не найден, 19 сравнений
91 ключ в дереве не найден, 19 сравнений
92 ключ в дереве не найден, 19 сравнений
93 ключ в дереве не найден, 19 сравнений
94 ключ в дереве не найден, 19 сравнений
95 ключ в дереве не найден, 19 сравнений
96 ключ в дереве не найден, 19 сравнений
97 ключ в дереве не найден, 19 сравнений
98 ключ в дереве не найден, 19 сравнений
99 ключ в дереве не найден, 19 сравнений
100 ключ в дереве не найден, 19 сравнений
```

Скриншот срабатывания программы при проверке только красно-чёрного дерева

Выводы:

Полученные результаты сошлись с теоретическими расчётами, в частности, для красно-чёрного дерева мы можем точно отследить лучший случай, который полностью совпал с расчётами и средний случай, который незначительно отличается от высчитанного

нами по формуле. Для рандомизированного дерева, мы лучший случай не встретили, зато присутствуют значения, близкие к среднему случаю и к худшему случаю. Если же сравнивать трудоёмкость поиска элементов в этих деревьях, то тут можно заметить явное лидерство красно-чёрного дерева при поиске элемента, находящегося в дереве, однако можно заметить, что при поиске элемента, не находящегося в дереве, рандомизированное дерево дышит в затылок красно-чёрному, а при некоторых значениях даже конкретно обгоняет его, из-за чего разница в среднем поиске не такая уж и большая. Пространственная же сложность в данной реализации у деревьев одинаковая, просто немного отличается начинка самих узлов. Исходя из вышеперечисленных пунктов я бы отдал своё предпочтение красно-чёрному дереву, так как оно надежнее в плане реализации и не зависит от “случая”, то есть, зная последовательность входящих элементов, вы всегда можете предсказать итоговый результат, в отличие от непредсказуемого рандомизированного дерева.